

# Kompendium projektu

Podsumowania lingwistyczne relacyjnej bazy samochodów

Teoria, miary jakości, kod, GUI i interpretacja wyników

Projekt linguistic-summaries

6 czerwca 2026

## Spis treści

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Jak czytać ten dokument</b>                            | <b>5</b>  |
| <b>1 Cel projektu w jednym obrazie</b>                    | <b>5</b>  |
| 1.1 Problem . . . . .                                     | 5         |
| 1.2 Główna forma zdania . . . . .                         | 6         |
| 1.3 Po co logika rozmyta . . . . .                        | 6         |
| <b>2 Dane i przepływ przetwarzania</b>                    | <b>6</b>  |
| 2.1 Źródło danych . . . . .                               | 6         |
| 2.2 Przepływ danych . . . . .                             | 7         |
| <b>3 Zbiory klasyczne i zbiory rozmyte</b>                | <b>7</b>  |
| 3.1 Zbiór klasyczny . . . . .                             | 7         |
| 3.2 Zbiór rozmyty . . . . .                               | 7         |
| 3.3 Uniwersum . . . . .                                   | 9         |
| 3.4 Funkcje przynależności . . . . .                      | 9         |
| 3.5 Wysokość, nośnik, alfa-przekrój, normalność . . . . . | 10        |
| 3.6 Operacje na zbiorach rozmytych . . . . .              | 10        |
| <b>4 Zmienne lingwistyczne i etykiety</b>                 | <b>11</b> |
| 4.1 Zmienna lingwistyczna . . . . .                       | 11        |
| 4.2 Etykieta lingwistyczna . . . . .                      | 11        |
| 4.3 Etykiety skonfigurowane w projekcie . . . . .         | 12        |
| <b>5 Kwantyfikatory</b>                                   | <b>12</b> |
| 5.1 Czym jest kwantyfikator . . . . .                     | 12        |

|           |                                                |           |
|-----------|------------------------------------------------|-----------|
| 5.2       | Kwantyfikator względny . . . . .               | 13        |
| 5.3       | Kwantyfikator absolutny . . . . .              | 13        |
| 5.4       | Kwantyfikator w kodzie . . . . .               | 14        |
| <b>6</b>  | <b>Sumaryzator, kwalifikator i podmiot</b>     | <b>14</b> |
| 6.1       | Sumaryzator . . . . .                          | 14        |
| 6.2       | Kwalifikator . . . . .                         | 15        |
| 6.3       | Podmiot . . . . .                              | 15        |
| <b>7</b>  | <b>Podsumowania jednopodmiotowe</b>            | <b>15</b> |
| 7.1       | Forma pierwsza . . . . .                       | 15        |
| 7.2       | Forma druga . . . . .                          | 16        |
| 7.3       | Spójniki AND i OR . . . . .                    | 16        |
| 7.4       | Obiekt podsumowania w kodzie . . . . .         | 16        |
| <b>8</b>  | <b>Stopień prawdziwości T1</b>                 | <b>17</b> |
| 8.1       | Dlaczego T1 jest najważniejsze . . . . .       | 17        |
| 8.2       | Forma I z kwantyfikatorem względnym . . . . .  | 17        |
| 8.3       | Forma I z kwantyfikatorem absolutnym . . . . . | 18        |
| 8.4       | Forma II z kwalifikatorem . . . . .            | 18        |
| <b>9</b>  | <b>Miary jakości T1–T11</b>                    | <b>19</b> |
| 9.1       | Po co tyle miar . . . . .                      | 19        |
| 9.2       | Tabela miar . . . . .                          | 19        |
| 9.3       | T1: prawdziwość . . . . .                      | 19        |
| 9.4       | T2: nieprecyzyjność sumaryzatora . . . . .     | 20        |
| 9.5       | T3: stopień pokrycia . . . . .                 | 20        |
| 9.6       | T4: stopień trafności . . . . .                | 20        |
| 9.7       | T5: długość podsumowania . . . . .             | 21        |
| 9.8       | T6 i T7: jakość kwantyfikatora . . . . .       | 21        |
| 9.9       | T8: kardynalność sumaryzatora . . . . .        | 21        |
| 9.10      | T9–T11: kwalifikator . . . . .                 | 21        |
| 9.11      | Wynik optymalny . . . . .                      | 22        |
| <b>10</b> | <b>Kardynalność i licznosc rozmyta</b>         | <b>22</b> |
| 10.1      | Dlaczego nie liczymy tylko rekordów . . . . .  | 22        |
| 10.2      | Sigma-count . . . . .                          | 22        |
| 10.3      | Względna kardynalność . . . . .                | 23        |

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>11 Podsumowania wielopodmiotowe</b>               | <b>23</b> |
| 11.1 Po co więcej niż jeden podmiot . . . . .        | 23        |
| 11.2 Podmioty P1 i P2 . . . . .                      | 23        |
| 11.3 Cztery formy wielopodmiotowe . . . . .          | 23        |
| 11.4 Stopień prawdziwości wielopodmiotowej . . . . . | 24        |
| 11.5 Kwalifikator w formach II i III . . . . .       | 24        |
| <b>12 Architektura kodu</b>                          | <b>26</b> |
| 12.1 Pakiety . . . . .                               | 26        |
| 12.2 Warstwa funkcji przynależności . . . . .        | 27        |
| 12.3 Warstwa zbiorów . . . . .                       | 27        |
| 12.4 Warstwa konfiguracji . . . . .                  | 27        |
| 12.5 Warstwa generowania zdań . . . . .              | 28        |
| 12.6 Warstwa oceny jakości . . . . .                 | 28        |
| 12.7 Warstwa GUI . . . . .                           | 28        |
| <b>13 GUI: jak działa aplikacja</b>                  | <b>29</b> |
| 13.1 Uruchomienie . . . . .                          | 29        |
| 13.2 Zakładka jednopodmiotowa . . . . .              | 29        |
| 13.3 Zakładka wielopodmiotowa . . . . .              | 30        |
| 13.4 Tryb zaawansowany . . . . .                     | 30        |
| 13.5 Eksport wyników . . . . .                       | 30        |
| <b>14 Jak czytać wygenerowane wyniki</b>             | <b>30</b> |
| 14.1 Nie patrz tylko na jedno pole . . . . .         | 30        |
| 14.2 Przykład interpretacyjny . . . . .              | 31        |
| 14.3 Interpretacja zdań z kwalifikatorem . . . . .   | 31        |
| 14.4 Interpretacja wielopodmiotowa . . . . .         | 31        |
| <b>15 Najważniejsze klasy: przewodnik po kodzie</b>  | <b>32</b> |
| 15.1 CarRecord . . . . .                             | 32        |
| 15.2 Columns . . . . .                               | 32        |
| 15.3 DataCleaner . . . . .                           | 32        |
| 15.4 FuzzyConfig . . . . .                           | 32        |
| 15.5 Property . . . . .                              | 32        |
| 15.6 LabelExpression . . . . .                       | 32        |
| 15.7 SummaryGenerator . . . . .                      | 32        |
| 15.8 QualityMeasures . . . . .                       | 33        |

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| 15.9 MultiSubjectGenerator i MultiSubjectMeasures . . . . .              | 33        |
| 15.10 SummaryGui . . . . .                                               | 33        |
| <b>16 Od zdania do wyniku: pełny przykład</b>                            | <b>33</b> |
| 16.1 Krok 1: wybór kwantyfikatora . . . . .                              | 34        |
| 16.2 Krok 2: wybór sumaryzatora . . . . .                                | 34        |
| 16.3 Krok 3: ocena każdego rekordu . . . . .                             | 34        |
| 16.4 Krok 4: agregacja . . . . .                                         | 34        |
| 16.5 Krok 5: zastosowanie kwantyfikatora . . . . .                       | 34        |
| 16.6 Krok 6: pozostałe miary . . . . .                                   | 35        |
| <b>17 Co zmienia konfiguracja funkcji przynależności</b>                 | <b>35</b> |
| 17.1 Granice słów są decyzją projektową . . . . .                        | 35        |
| 17.2 Dlaczego wykresy są ważne . . . . .                                 | 35        |
| <b>18 Testy i poprawność</b>                                             | <b>36</b> |
| 18.1 Co sprawdzają testy . . . . .                                       | 36        |
| 18.2 Co trzeba sprawdzać ręcznie . . . . .                               | 36        |
| <b>19 Najczęstsze pułapki</b>                                            | <b>36</b> |
| 19.1 Pomylenie kwantyfikatora względnego z absolutnym . . . . .          | 36        |
| 19.2 Traktowanie stopnia przynależności jak prawdopodobieństwa . . . . . | 37        |
| 19.3 Traktowanie T1 jako jedynej miary jakości . . . . .                 | 37        |
| 19.4 Zapominanie o kwalifikatorze . . . . .                              | 37        |
| 19.5 Mylenie form wielopodmiotowych . . . . .                            | 37        |
| <b>20 Ściąga pojęć</b>                                                   | <b>37</b> |
| <b>21 Ściąga po klasach</b>                                              | <b>37</b> |
| <b>22 Jak wytłumaczyć projekt komuś na głos</b>                          | <b>38</b> |
| <b>23 Bibliografia orientacyjna</b>                                      | <b>39</b> |
| <b>Zakończenie</b>                                                       | <b>39</b> |

# Jak czytać ten dokument

Ten plik jest podręcznikiem do całego projektu. Nie zastępuje sprawozdania oddawanego prowadzącemu, tylko tłumaczy projekt tak, żeby dało się go naprawdę zrozumieć: od idei zbioru rozmytego, przez kwantyfikator, sumaryzator i kwalifikator, aż po konkretne klasy Javy, miary jakości i obsługę aplikacji okienkowej.

Najważniejsza myśl przewodnia jest prosta: projekt zamienia surową tabelę rekordów samochodów na zdania w języku naturalnym typu:

*Większość samochodów ma małą pojemność bagażnika.*

Takie zdanie nie jest oceniane jako tylko prawda albo fałsz. Ono dostaje stopień prawdziwości oraz dodatkowe miary jakości, bo w logice rozmytej pojęcia takie jak *mały*, *duży*, *szybki*, *większość* albo *około połowy* nie mają ostrych granic.

W dokumencie konsekwentnie rozróżniamy trzy warstwy:

- **teorię**: zbiory rozmyte, funkcje przynależności, zmienne lingwistyczne, kwantyfikatory, formy podsumowań;
- **algorytm**: jak z rekordów, etykiet i kwantyfikatorów powstają zdania oraz miary T1–T11;
- **kod**: które klasy za co odpowiadają i jak przepływ danych przechodzi od CSV/SQLite do GUI i eksportu wyników.

## 1 Cel projektu w jednym obrazie

### 1.1 Problem

Mamy relacyjną bazę danych o samochodach. Każdy rekord opisuje jeden model lub wariant samochodu, a kolumny zawierają atrybuty liczbowe oraz nominalne, między innymi markę, rok produkcji, wymiary, pojemność silnika, moc, moment obrotowy, spalanie, przyspieszenie i prędkość maksymalną.

Klasyczna analiza mogłaby dać tabelę średnich:

| Kolumna                             | Przykładowa statystyka klasyczna |
|-------------------------------------|----------------------------------|
| engine_hp                           | średnia moc silnika              |
| max_speed_km_per_h                  | średnia prędkość maksymalna      |
| mixed_fuel_consumption_per_100_km_l | mediana spalania                 |

Podsumowania lingwistyczne robią coś innego: opisują dane zdaniami bliższymi temu, jak człowiek mówi o świecie. Zamiast pytać tylko „jaka jest średnia moc?”, pytamy:

- Czy *większość samochodów* ma *małą pojemność silnika*?
- Czy *około połowy samochodów* jest *szybka i ekonomiczna*?
- Czy wśród samochodów z *dużym bagażnikiem* *większość* ma *dużą masę własną*?
- Czy samochody marki BMW częściej niż Toyota mają *dużą moc silnika*?

## 1.2 Główna forma zdania

Najprostsze podsumowanie ma postać:

$$Q \ Y \text{ jest/są } S$$

gdzie:

- $Y$  to zbiór obiektów, czyli w projekcie najczęściej *samochody*;
- $Q$  to **kwantyfikator**, np. *większość*, *prawie żaden*, *kilkanaście tysięcy*;
- $S$  to **summaryzator**, czyli właściwość przypisywana obiektom, np. *ma małą pojemność silnika*;
- całe zdanie jest oceniane liczbowo w przedziale  $[0, 1]$ .

Przykład:

Większość samochodów ma małą pojemność silnika.

Tutaj  $Q = \text{większość}$ ,  $Y = \text{samochody}$ ,  $S = \text{ma małą pojemność silnika}$ .

## 1.3 Po co logika rozmyta

W klasycznej logice predykat *samochód ma małą pojemność silnika* musiałby mieć twardą granicę. Na przykład:

$$\text{mała pojemność} = \begin{cases} 1, & \text{gdy pojemność} \leq 1400 \\ 0, & \text{gdy pojemność} > 1400 \end{cases}$$

To prowadzi do sztucznych skoków. Silnik 1400 cm<sup>3</sup> byłby mały, a 1401 cm<sup>3</sup> już nie. Logika rozmyta mówi: można należeć do pojęcia *mały* częściowo. Silnik 1200 cm<sup>3</sup> może należeć do etykiety *mała pojemność* w stopniu 1.0, silnik 1600 cm<sup>3</sup> w stopniu 0.6, a silnik 2500 cm<sup>3</sup> w stopniu 0.0.

W tym projekcie prawdziwość zdania nie wynika z jednego rekordu, tylko z agregacji stopni przynależności wielu rekordów do rozmytych pojęć.

## 2 Dane i przepływ przetwarzania

### 2.1 Źródło danych

Projekt korzysta ze zbioru *Car Specification Dataset 1945–2020*. Dane są trzymane w repozytorium jako CSV, a następnie importowane do SQLite. Po oczyszczeniu używany jest zbiór filtrowany, zawierający 23441 rekordów i kolumny potrzebne do generowania podsumowań.

W praktyce interesują nas głównie kolumny numeryczne, bo to dla nich definiujemy zmienne lingwistyczne:

| Kolumna        | Znaczenie                       |
|----------------|---------------------------------|
| length_mm      | długość samochodu w milimetrach |
| wheelbase_mm   | rozstaw osi w milimetrach       |
| curb_weight_kg | masa własna w kilogramach       |

| Kolumna                                          | Znaczenie                                   |
|--------------------------------------------------|---------------------------------------------|
| <code>minimum_trunk_capacity_l</code>            | minimalna pojemność bagażnika w litrach     |
| <code>maximum_torque_n_m</code>                  | maksymalny moment obrotowy w niutonometrach |
| <code>capacity_cm3</code>                        | pojemność skokowa silnika w cm <sup>3</sup> |
| <code>engine_hp</code>                           | moc silnika w koniach mechanicznych         |
| <code>turning_circle_m</code>                    | średnica zawracania w metrach               |
| <code>mixed_fuel_consumption_per_100_km_l</code> | spalanie mieszane w litrach na 100 km       |
| <code>fuel_tank_capacity_l</code>                | pojemność zbiornika paliwa w litrach        |
| <code>acceleration_0_100_s</code>                | przyspieszenie 0–100 km/h w sekundach       |
| <code>max_speed_km_per_h</code>                  | prędkość maksymalna w km/h                  |

## 2.2 Przepływ danych

Przepływ danych w aplikacji wygląda następująco:

1. Plik CSV zawiera surowe dane o samochodach.
2. `DbImporter` importuje dane do SQLite.
3. `DataCleaner` filtruje dane i usuwa rekordy nieprzydatne do analizy.
4. `CarDataLoader` ładuje rekordy do obiektów `CarRecord`.
5. `FuzzyConfig` tworzy zmienne lingwistyczne, etykiety i kwantyfikatory.
6. `SummaryGenerator` lub `MultiSubjectGenerator` tworzy podsumowania.
7. `QualityMeasures` albo `MultiSubjectMeasures` liczy jakość.
8. `SummaryGui` pokazuje wyniki, pozwala sortować, dobierać wagi i eksportować CSV.

## 3 Zbiory klasyczne i zbiory rozmyte

### 3.1 Zbiór klasyczny

W zbiorze klasycznym obiekt albo należy do zbioru, albo nie należy. Funkcja charakterystyczna ma tylko wartości 0 lub 1:

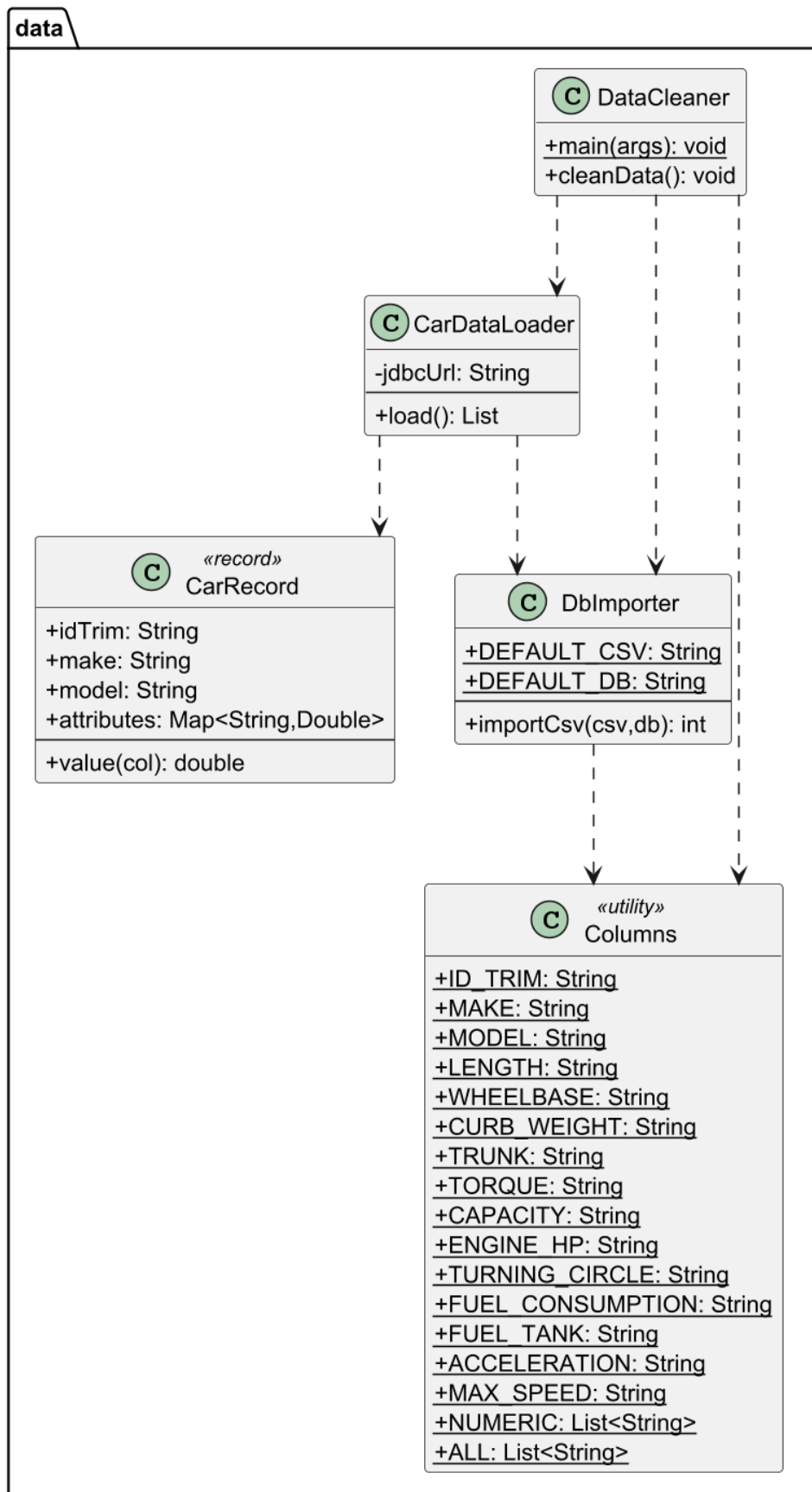
$$\chi_A(x) = \begin{cases} 1, & x \in A \\ 0, & x \notin A \end{cases}$$

Jeżeli zdefiniujemy klasycznie *samochody szybkie* jako te z prędkością maksymalną co najmniej 200 km/h, to auto z prędkością 199 km/h nie jest szybkie, a auto z prędkością 200 km/h jest szybkie. To jest proste, ale często zbyt brutalne.

### 3.2 Zbiór rozmyty

Zbiór rozmyty  $A$  opisuje funkcja przynależności:

$$\mu_A : X \rightarrow [0, 1]$$



Rysunek 1: Warstwa danych: rekordy, import, czyszczenie i ładowanie.



Wartość  $\mu_A(x)$  oznacza, w jakim stopniu element  $x$  należy do zbioru  $A$ :

- 0: element w ogóle nie należy;
- 1: element należy w pełni;
- wartości między 0 i 1: element należy częściowo.

Przykład:

$$\mu_{\text{szybki}}(190) = 0.4, \quad \mu_{\text{szybki}}(210) = 0.8, \quad \mu_{\text{szybki}}(250) = 1.0$$

Takie podejście jest naturalne, bo wiele pojęć językowych ma granice płynne, nie ostre.

### 3.3 Uniwersum

**Uniwersum** to dziedzina wartości, na której działa zbiór rozmyty. Dla prędkości maksymalnej może to być np. przedział od 0 do 400 km/h. W kodzie odpowiada za to klasa **Universe**.

W projekcie uniwersum potrafi być:

- **ciągłe**: np. prędkość, masa, długość;
- **dyskretne**: gdyby analizowane wartości były skończonym zestawem punktów.

Klasa **Universe** udostępnia między innymi punkty próbkowania. Są potrzebne, ponieważ wiele własności zbiorów rozmytych, np. nośnik albo kardynalność, jest w projekcie liczona numerycznie przez przejście po próbkach dziedziny.

### 3.4 Funkcje przynależności

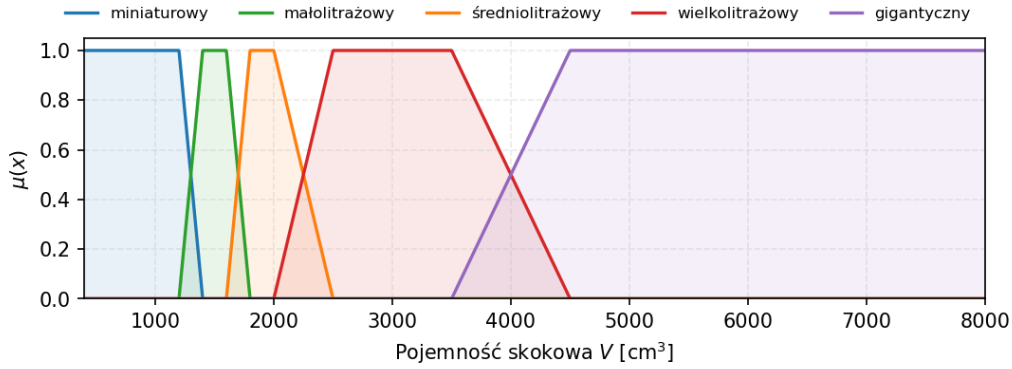
Funkcja przynależności mówi, jak wartość liczbową zamienia się w stopień spełnienia etykiety. W projekcie są zaimplementowane trzy typy funkcji:

| Funkcja       | Parametry    | Intuicja                                                |
|---------------|--------------|---------------------------------------------------------|
| trójkątna     | $a, b, c$    | rośnie liniowo do maksimum w $b$ , potem liniowo maleje |
| trapezoidalna | $a, b, c, d$ | ma lewy stok, plateau z wartością 1 i prawy stok        |
| gaussowska    | $c, \sigma$  | gładki dzwon wokół środka $c$                           |

Kodowo odpowiadają im klasy:

- **TriangularMf**;
- **TrapezoidalMf**;
- **GaussianMf**.

Najczęściej używana jest funkcja trapezoidalna, bo dobrze modeluje etykiety językowe typu *mały*, *średni*, *duży*: istnieje zakres, w którym etykieta pasuje w pełni, oraz zakresy przejściowe.



Rysunek 2: Przykładowe etykiety zmiennej lingwistycznej dla pojemności silnika.

### 3.5 Wysokość, nośnik, alfa-przekrój, normalność

Dla zbioru rozmytego ważne są pojęcia pomocnicze:

**Wysokość** największa wartość funkcji przynależności:

$$h(A) = \sup_{x \in X} \mu_A(x)$$

Jeżeli wysokość wynosi 1, zbiór gdzieś osiąga pełną przynależność.

**Nośnik** zbiór wartości, dla których przynależność jest dodatnia:

$$\text{supp}(A) = \{x \in X : \mu_A(x) > 0\}$$

**Alfa-przekrój** zbiór wartości, których przynależność jest co najmniej  $\alpha$ :

$$A_\alpha = \{x \in X : \mu_A(x) \geq \alpha\}$$

**Normalność** zbiór jest normalny, jeżeli jego wysokość wynosi 1.

**Pustość** zbiór jest pusty, gdy nigdzie nie ma dodatniej przynależności.

**Wypukłość rozmyta** intuicyjnie: przekroje alfa są spójne; dla typowych etykiet trójkątnych i trapezoidalnych jest to spełnione.

Klasa `FuzzySet` obsługuje te operacje przez próbkowanie uniwersum.

### 3.6 Operacje na zbiorach rozmytych

W projekcie używane są standardowe operacje Zadeha:

$$\mu_{\neg A}(x) = 1 - \mu_A(x)$$

$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$$

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$$

To ma bezpośredni związek z językiem:

- *mały i lekki* oznacza minimum z dwóch stopni;
- *mały lub lekki* oznacza maksimum z dwóch stopni;
- *nie mały* oznacza dopełnienie.

W kodzie operacje są w `FuzzySet` oraz w pomocniczej klasie `Operations`. Łączniki zdań są opisane przez enum `Connective`: AND używa minimum, a OR maksimum.

## 4 Zmienne lingwistyczne i etykiety

### 4.1 Zmienna lingwistyczna

**Zmienna lingwistyczna** to atrybut, którego wartości opisujemy słowami. Formalnie można myśleć o niej jako o zestawie:

$\langle \text{nazwa, dziedzina, etykiety} \rangle$

W projekcie przykładem zmiennej lingwistycznej jest:

*pojemność silnika*

Jej dziedziną są wartości w  $\text{cm}^3$ , a etykietami mogą być np.:

- *bardzo mała pojemność*;
- *mała pojemność*;
- *średnia pojemność*;
- *duża pojemność*;
- *bardzo duża pojemność*.

W kodzie zmienna lingwistyczna to `LinguisticVariable`. Przechowuje:

- nazwę wyświetlaną użytkownikowi;
- nazwę kolumny w rekordzie;
- uniwersum;
- listę etykiet;
- metodę `degreeOf`, która liczy stopień przynależności wartości do wybranej etykiety.

### 4.2 Etykieta lingwistyczna

**Etykieta** to pojedyncze słowne pojęcie na zmiennej lingwistycznej. Na przykład dla zmiennej *moc silnika* etykietą może być *duża moc*. Etykieta ma nazwę i zbiór rozmyty, czyli funkcję przynależności.

W kodzie etykieta to rekord `Label`. Jej znaczenie jest krótkie, ale fundamentalne: bez etykiet nie ma sumaryzatorów ani kwalifikatorów.

### 4.3 Etykiety skonfigurowane w projekcie

Każda z 12 zmiennych liczbowych ma po 5 etykiet trapezoidalnych. Dzięki temu aplikacja może tworzyć wiele zdań automatycznie.

| Zmienna                    | Typowe etykiety                                               |
|----------------------------|---------------------------------------------------------------|
| długość samochodu          | bardzo krótki, krótki, średniej długości, długi, bardzo długi |
| rozstaw osi                | bardzo mały, mały, średni, duży, bardzo duży                  |
| masa własna                | bardzo lekki, lekki, średniej masy, ciężki, bardzo ciężki     |
| pojemność bagażnika        | bardzo mała, mała, średnia, duża, bardzo duża                 |
| moment obrotowy            | bardzo mały, mały, średni, duży, bardzo duży                  |
| pojemność silnika          | bardzo mała, mała, średnia, duża, bardzo duża                 |
| moc silnika                | bardzo mała, mała, średnia, duża, bardzo duża                 |
| średnica zawracania        | bardzo mała, mała, średnia, duża, bardzo duża                 |
| spalanie mieszane          | bardzo niskie, niskie, średnie, wysokie, bardzo wysokie       |
| pojemność zbiornika paliwa | bardzo mała, mała, średnia, duża, bardzo duża                 |
| przyspieszenie 0–100       | bardzo szybkie, szybkie, średnie, wolne, bardzo wolne         |
| prędkość maksymalna        | bardzo mała, mała, średnia, duża, bardzo duża                 |

Nazwy etykiet są ważne nie tylko matematycznie, ale też językowo, bo trafiają bezpośrednio do generowanych zdań.

## 5 Kwantyfikatory

### 5.1 Czym jest kwantyfikator

**Kwantyfikator** mówi, jak dużej części zbioru dotyczy zdanie. W klasycznej logice mamy kwantyfikatory ostre, np.:

- *dla każdego;*
- *istnieje;*
- *co najmniej jeden.*

W podsumowaniach lingwistycznych używamy kwantyfikatorów rozmytych:

- *prawie żaden;*
- *mniejść;*
- *około połowy;*
- *większość;*
- *prawie wszystkie;*
- *kilka tysięcy;*
- *kilkanaście tysięcy;*

- *dużo*.

Kwantyfikator też jest zbiorem rozmytym, tylko jego dziedziną nie jest np. prędkość samochodu, lecz liczność albo proporcja.

## 5.2 Kwantyfikator względny

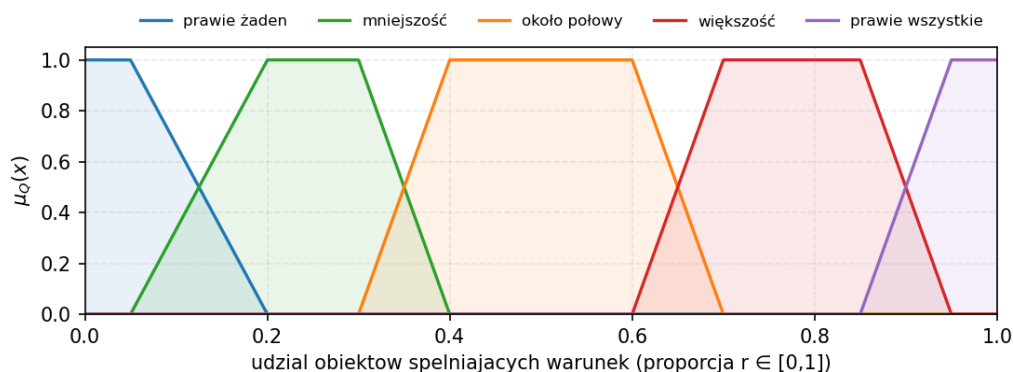
Kwantyfikator względny działa na proporcji z przedziału  $[0, 1]$ . Na przykład:

udział samochodów spełniających  $S = 0.72$

Wtedy pytamy, w jakim stopniu 0.72 pasuje do kwantyfikatora *większość*.

W projekcie skonfigurowano:

| Kwantyfikator    | Parametry trapezu      | Typ      |
|------------------|------------------------|----------|
| prawie żaden     | 0, 0, 0.05, 0.20       | względny |
| mniejszość       | 0.05, 0.20, 0.30, 0.40 | względny |
| około połowy     | 0.30, 0.40, 0.60, 0.70 | względny |
| większość        | 0.60, 0.70, 0.85, 0.95 | względny |
| prawie wszystkie | 0.85, 0.95, 1, 1       | względny |



Rysunek 3: Kwantyfikatory względne używane w projekcie.

## 5.3 Kwantyfikator absolutny

Kwantyfikator absolutny działa na liczności, a nie na proporcji. Przykład:

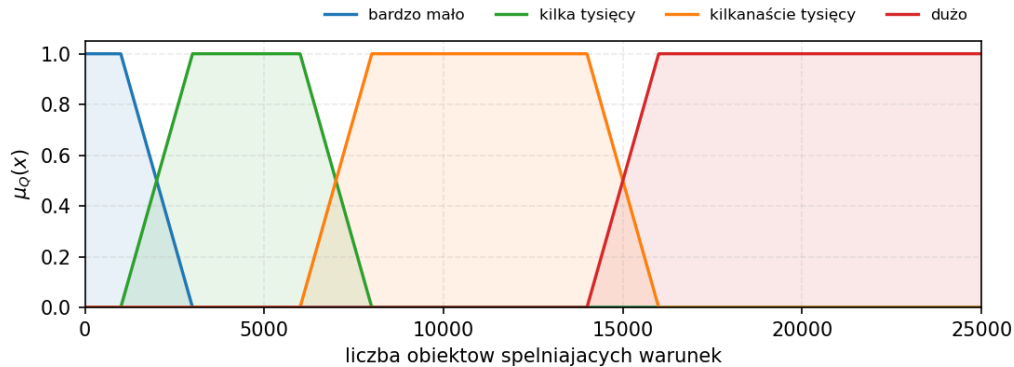
sumaryczna liczba samochodów spełniających  $S = 7800$

Wtedy sprawdzamy, jak 7800 pasuje do kwantyfikatora *kilka tysięcy* albo *kilkanaście tysięcy*.

W projekcie skonfigurowano:

| Kwantyfikator | Parametry trapezu      | Typ       |
|---------------|------------------------|-----------|
| bardzo mało   | 0, 0, 1000, 3000       | absolutny |
| kilka tysięcy | 1000, 3000, 6000, 8000 | absolutny |

| Kwantyfikator       | Parametry trapezu          | Typ       |
|---------------------|----------------------------|-----------|
| kilkanaście tysięcy | 6000, 8000, 14000, 16000   | absolutny |
| dużo                | 14000, 16000, 25000, 25000 | absolutny |



Rysunek 4: Kwantyfikatory absolutne używane w projekcie.

## 5.4 Kwantyfikator w kodzie

W kodzie kwantyfikator to `Quantifier`. Przechowuje nazwę, typ i zbiór rozmyty. Najważniejsza metoda to `degree(quantity)`, czyli odpowiedź na pytanie:

*W jakim stopniu ta proporcja albo ta liczba pasuje do tego kwantyfikatora?*

Jeżeli kwantyfikator jest względny, argument powinien być proporcją. Jeżeli jest absolutny, argument powinien być licznoscia. Poprawny dobór argumentu jest istotny, bo *większość* nie ma sensu dla liczby 12000, a *kilka tysięcy* nie ma sensu dla wartości 0.6.

# 6 Sumaryzator, kwalifikator i podmiot

## 6.1 Sumaryzator

**Sumaryzator** to właściwość, którą przypisujemy obiektom. W zdaniu:

*Większość samochodów ma dużą moc silnika.*

sumaryzatorem jest:

*ma dużą moc silnika*

Sumaryzator może być prosty, gdy składa się z jednej etykiety, albo złożony, gdy łączy dwie etykiety spójnikiem:

- prosty: *ma dużą moc silnika*;
- złożony z *i*: *ma dużą moc silnika i niskie spalanie*;
- złożony z *lub*: *ma dużą moc silnika lub dużą prędkość maksymalną*.

Kodowo atomowym sumaryzatorem jest `Property`. To obiekt łączący zmienną lingwistyczną z etykietą. `Property.degree(record)` pobiera z rekordu odpowiednią kolumnę i liczy stopień przynależności wartości do etykiety.

## 6.2 Kwalifikator

**Kwalifikator** zawęża zbiór obiektów, o których mówimy. Zdanie z kwalifikatorem ma postać:

$$Q\ Y\ \text{będących}\ R\ \text{jest/są}\ S$$

Przykład:

*Większość samochodów o dużym bagażniku ma dużą masę własną.*

Tutaj:

- $Q = \text{większość}$ ;
- $Y = \text{samochody}$ ;
- $R = \text{o dużym bagażniku}$ ;
- $S = \text{ma dużą masę własną}$ .

Kwalifikator działa jak filtr, ale filtr rozmyty. Samochód może należeć do grupy *o dużym bagażniku* w stopniu 0.2, 0.7 albo 1.0. Dlatego w obliczeniach nie wyrzucamy rekordów zero-jedynkowo, tylko ważymy je stopniem spełnienia kwalifikatora.

## 6.3 Podmiot

W podsumowaniach jednopodmiotowych podmiotem jest cały zbiór samochodów. W podsumowaniach wielopodmiotowych podmiotami są grupy samochodów, np. marki:

- samochody marki BMW;
- samochody marki Toyota;
- samochody marki Ford.

Kodowo podmiot wielopodmiotowy to `Subject`. Najczęściej powstaje przez `Subject.ofMake(make)`, czyli przez predykat wybierający rekordy o konkretnej marce.

## 7 Podsumowania jednopodmiotowe

### 7.1 Forma pierwsza

Forma pierwsza nie ma kwalifikatora:

$$Q\ Y\ \text{jest/są}\ S$$

Przykłady:

- *Większość samochodów ma małą pojemność silnika.*

- *Okolo połowy samochodów ma średnią prędkość maksymalną.*
- *Kilka tysięcy samochodów ma bardzo duży moment obrotowy.*

W kodzie forma pierwsza powstaje metodami:

- `SummaryGenerator.simpleForm1` dla pojedynczego sumaryzatora;
- `SummaryGenerator.compoundForm1` dla dwóch sumaryzatorów połączonych AND albo OR.

## 7.2 Forma druga

Forma druga ma kwalifikator:

$Q\ Y\ \text{będących}\ R\ \text{jest/są}\ S$

Przykłady:

- *Większość samochodów o dużej mocy silnika ma dużą prędkość maksymalną.*
- *Mniejszość samochodów o niskim spalaniu ma bardzo dużą masę własną.*
- *Okolo połowy samochodów z dużym bagażnikiem ma duży rozstaw osi.*

W kodzie forma druga powstaje metodą:

- `SummaryGenerator.simpleForm2`.

Projekt pilnuje, żeby kwalifikator i sumaryzator nie były tą samą prostą właściwością. Zdanie typu *Większość samochodów będących szybkimi jest szybka* byłoby mało informacyjne.

## 7.3 Spójniki AND i OR

Dla złożonych sumaryzatorów projekt używa:

$$\mu_{S_1 \wedge S_2}(y) = \min(\mu_{S_1}(y), \mu_{S_2}(y))$$

$$\mu_{S_1 \vee S_2}(y) = \max(\mu_{S_1}(y), \mu_{S_2}(y))$$

Czyli:

- *szybki i ekonomiczny* jest tak mocne, jak słabsza z dwóch cech;
- *szybki lub ekonomiczny* jest tak mocne, jak mocniejsza z dwóch cech.

To jest jeden z najważniejszych miejsc, w których matematyka zbiorów rozmytych bezpośrednio przekłada się na język.

## 7.4 Obiekt podsumowania w kodzie

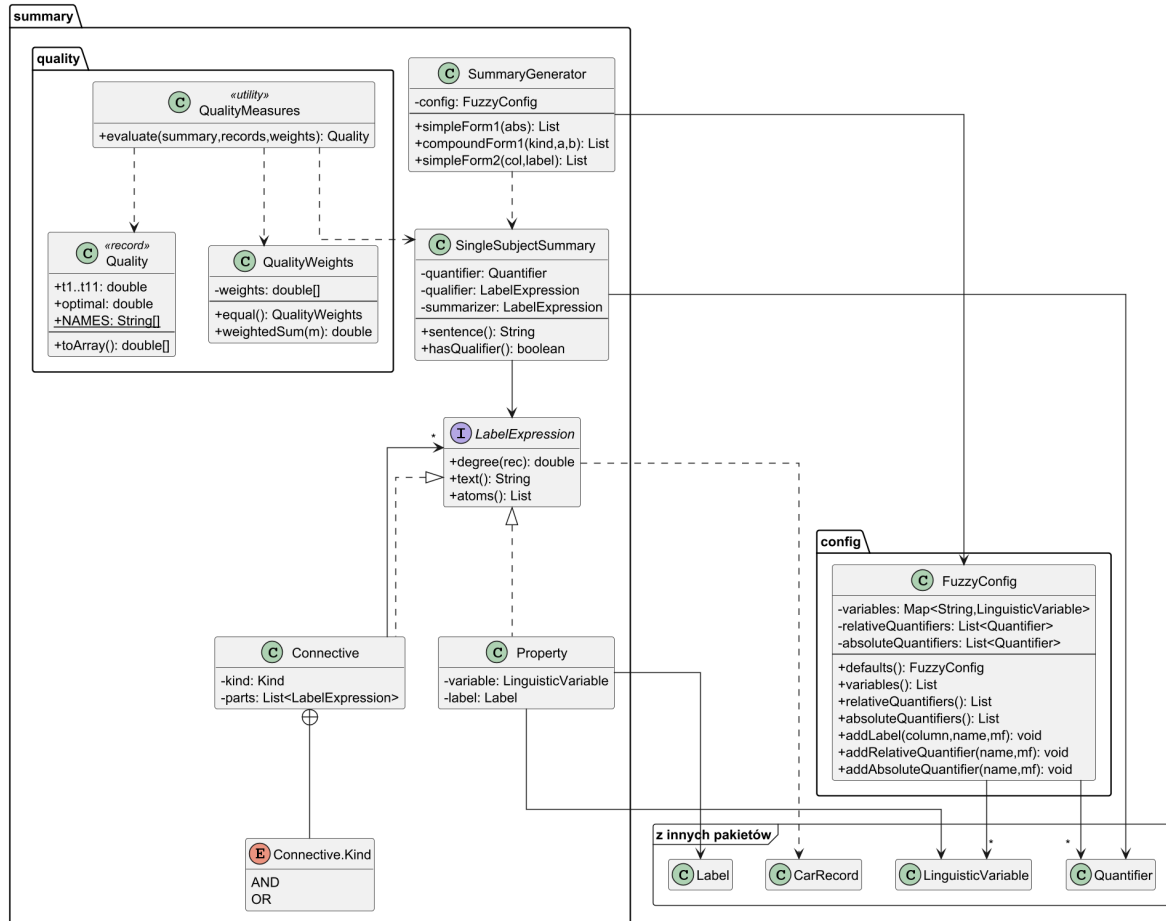
Wynikiem generatora jest `SingleSubjectSummary`. Ten obiekt przechowuje:

- formę: I albo II;
- kwantyfikator;



- sumaryzator;
- opcjonalny kwalifikator;
- tekst zdania;
- typ kwantyfikatora: względny albo absolutny.

Miary jakości nie są częścią samego zdania. Są liczone osobno przez `QualityMeasures.evaluate`.



Rysunek 5: Kluczowe klasy dla podsumowań jednopodmiotowych.

## 8 Stopień prawdziwości T1

### 8.1 Dlaczego T1 jest najważniejsze

Miara T1 to **stopień prawdziwości podsumowania**. Odpowiada na pytanie:

*Na ile dane rzeczywiście potwierdzają wygenerowane zdanie?*

To jest najbliższy odpowiednik prawdy logicznej, ale w przedziale  $[0, 1]$ .

### 8.2 Forma I z kwantyfikatorem względnym

Dla zdania:

$Q$   $Y$  jest/są  $S$

liczymy średni stopień spełnienia sumaryzatora wśród obiektów:

$$r = \frac{1}{n} \sum_{i=1}^n \mu_S(y_i)$$

Następnie sprawdzamy, jak ta proporcja pasuje do kwantyfikatora:

$$T_1 = \mu_Q(r)$$

Przykład intuicyjny: jeżeli średni stopień spełnienia etykiety *mała pojemność silnika* wynosi 0.72, to dla kwantyfikatora *większość* wynik może być wysoki.

### 8.3 Forma I z kwantyfikatorem absolutnym

Dla kwantyfikatora absolutnego nie dzielimy przez liczbę rekordów. Liczymy rozmytą licznosc:

$$c = \sum_{i=1}^n \mu_S(y_i)$$

a potem:

$$T_1 = \mu_Q(c)$$

Przykład: jeżeli suma przynależności do *dużej mocy* wynosi 7500, to sprawdzamy, czy 7500 pasuje do *kilku tysięcy* albo *kilkunastu tysięcy*.

### 8.4 Forma II z kwalifikatorem

Dla formy:

$Q$   $Y$  będących  $R$  jest/są  $S$

kwalifikator ogranicza zbiór obiektów. W wersji względnej liczymy:

$$r = \frac{\sum_{i=1}^n \min(\mu_R(y_i), \mu_S(y_i))}{\sum_{i=1}^n \mu_R(y_i)}$$

i następnie:

$$T_1 = \mu_Q(r)$$

W liczniku jest część kwalifikowanych obiektów, która spełnia też sumaryzator. W mianowniku jest rozmyta licznosc obiektów kwalifikowanych.

Jeżeli mianownik wynosi 0, to znaczy, że kwalifikator nie opisuje żadnego obiektu w danych. Projekt zwraca wtedy 0, bo nie ma podstaw do uznania zdania za prawdziwe.

## 9 Miary jakości T1–T11

### 9.1 Po co tyle miar

Samo T1 nie wystarcza. Zdanie może być prawdziwe, ale mało przydatne. Na przykład:

*Prawie wszystkie samochody mają jakąś długość.*

Mogłoby być bardzo prawdziwe, ale informacyjnie słabe. Dlatego system Yagera/Kacprzyka używa dodatkowych miar: część ocenia prawdziwość, część specyficzność, część prostotę, część informacyjność kwantyfikatora i kwalifikatora.

W projekcie klasa `QualityMeasures` liczy 11 miar i wynik optymalny:

$$OPT = \sum_{i=1}^{11} w_i T_i$$

gdzie  $w_i$  to waga miary  $T_i$ . Wagi są walidowane i normalizowane przez `QualityWeights`.

### 9.2 Tabela miar

| Miara | Nazwa intuicyjna               | Co mierzy                                                                     |
|-------|--------------------------------|-------------------------------------------------------------------------------|
| T1    | prawdziwość                    | zgodność zdania z danymi po zastosowaniu kwantyfikatora                       |
| T2    | nieprecyzyjność sumaryzatora   | im szersze etykiety sumaryzatora, tym mniejsza wartość tej miary              |
| T3    | stopień pokrycia               | jaki odsetek obiektów dodatnio spełnia sumaryzator                            |
| T4    | stopień trafności              | różnica między pokryciem rzeczywistym a oczekiwanym przy niezależności atomów |
| T5    | długość podsumowania           | premiuje krótsze, prostsze sumaryzatory                                       |
| T6    | nieprecyzyjność kwantyfikatora | premiuje bardziej konkretne kwantyfikatory                                    |
| T7    | kardynalność kwantyfikatora    | premiuje kwantyfikatory o mniejszej rozmytej liczności                        |
| T8    | kardynalność sumaryzatora      | premiuje bardziej selektywne sumaryzatory                                     |
| T9    | nieprecyzyjność kwalifikatora  | analog T2 dla kwalifikatora, tylko w formie II                                |
| T10   | kardynalność kwalifikatora     | analog T8 dla kwalifikatora, tylko w formie II                                |
| T11   | długość kwalifikatora          | premiuje krótsze kwalifikatory, tylko w formie II                             |

### 9.3 T1: prawdziwość

T1 opisano szczegółowo w poprzednim rozdziale. To miara najłatwiejsza do intuicyjnej interpretacji: mówi, czy zdanie pasuje do danych.

W kodzie T1 zależy od:

- formy podsumowania;
- typu kwantyfikatora;
- sumy stopni przynależności do sumaryzatora;
- w formie II także od sumy stopni kwalifikatora.

#### 9.4 T2: nieprecyzyjność sumaryzatora

T2 ocenia, jak szeroki jest sumaryzator. Jeśli etykieta obejmuje bardzo dużą część uniwersum, to jest mało precyzyjna. Jeżeli obejmuje niewielki obszar, jest bardziej konkretna.

W projekcie używana jest względna kardynalność nośnika etykiety:

$$T_2 = 1 - \sqrt[m]{\prod_{j=1}^m \text{suppRel}(S_j)}$$

gdzie  $m$  to liczba atomowych właściwości w sumaryzatorze.

Interpretacja:

- wysoka T2: sumaryzator jest precyzyjny;
- niska T2: sumaryzator jest szeroki i ogólny.

#### 9.5 T3: stopień pokrycia

T3 sprawdza, ile obiektów jest dodatnio objętych przez sumaryzator.

Dla formy I:

$$T_3 = \frac{|\{y_i : \mu_S(y_i) > 0\}|}{n}$$

Dla formy II:

$$T_3 = \frac{|\{y_i : \mu_R(y_i) > 0 \wedge \mu_S(y_i) > 0\}|}{|\{y_i : \mu_R(y_i) > 0\}|}$$

T3 używa dodatniej przynależności, więc nie patrzy na to, czy stopień wynosi 0.1 czy 0.9. Patrzy tylko, czy obiekt w ogóle został objęty.

#### 9.6 T4: stopień trafności

T4 porównuje rzeczywiste pokrycie sumaryzatora z pokryciem oczekiwanym na podstawie pojedynczych atomów. Dla złożonych sumaryzatorów pozwala ocenić, czy kombinacja cech daje coś więcej niż przypadkowe nałożenie szerokich etykiet.

W kodzie:

$$T_4 = \left| \prod_{j=1}^m p_j - T_3 \right|$$

gdzie  $p_j$  to udział rekordów dodatnio spełniających atom  $S_j$ .

## 9.7 T5: długość podsumowania

T5 premiuje krótkie podsumowania. W projekcie:

$$T_5 = 2 \cdot \left(\frac{1}{2}\right)^m$$

Dla jednego atomu:

$$T_5 = 1$$

Dla dwóch atomów:

$$T_5 = 0.5$$

To znaczy, że proste zdanie z jednym sumaryzatorem dostaje maksymalną premię za długość, a zdanie złożone jest karane, bo jest mniej zwarte.

## 9.8 T6 i T7: jakość kwantyfikatora

T6 i T7 dotyczą kwantyfikatora.

T6 mierzy nieprecyzyjność przez nośnik:

$$T_6 = 1 - \text{suppRel}(Q)$$

T7 mierzy rozmytą kardynalność:

$$T_7 = 1 - \text{sigmaRel}(Q)$$

Im kwantyfikator węższy i bardziej konkretny, tym wyższe te miary. *Okolo połowy* może być dość informacyjne, bo wskazuje środek skali. Bardzo szeroki kwantyfikator byłby mniej użyteczny.

## 9.9 T8: kardynalność sumaryzatora

T8 jest podobna do T2, ale używa rozmytej kardynalności, czyli sumy stopni przynależności, a nie tylko szerokości nośnika:

$$T_8 = 1 - \sqrt[m]{\prod_{j=1}^m \text{sigmaRel}(S_j)}$$

Wysoka T8 oznacza, że sumaryzator jest selektywny w sensie rozmytym.

## 9.10 T9–T11: kwalifikator

T9, T10 i T11 pojawiają się tylko dla formy II. Jeżeli podsumowanie nie ma kwalifikatora, projekt ustawia je na 0.

- T9: nieprecyzyjność kwalifikatora;
- T10: rozmyta kardynalność kwalifikatora;

- T11: długość kwalifikatora.

Ich sens jest analogiczny do T2, T8 i T5, tylko dotyczą warunku zawężającego zbiór obiektów.

## 9.11 Wynik optymalny

Wynik optymalny to średnia ważona:

$$OPT = w_1T_1 + w_2T_2 + \dots + w_{11}T_{11}$$

Domyślne wagi mogą sprawić, że zdanie o niezbyt wysokim T1 nadal będzie wysoko w rankingu, jeżeli ma dobry kwantyfikatory, precyzyjny sumaryzator i odpowiednią długość. Dlatego podczas interpretacji wyników trzeba zawsze patrzeć na:

- T1, gdy interesuje nas prawdziwość;
- OPT, gdy interesuje nas kompromis jakości;
- T2/T8, gdy chcemy wiedzieć, czy sumaryzator jest konkretny;
- T3, gdy chcemy rozumieć pokrycie danych;
- T9–T11, gdy analizujemy zdania z kwalifikatorem.

## 10 Kardynalność i liczność rozmyta

### 10.1 Dlaczego nie liczymy tylko rekordów

W klasycznym świecie liczność zbioru to liczba elementów. W świecie rozmytym rekord może należeć do zbioru częściowo. Jeśli mamy trzy samochody o stopniach przynależności do etykiety *szybki*:

$$0.2, \quad 0.8, \quad 1.0$$

to rozmyta liczność wynosi:

$$0.2 + 0.8 + 1.0 = 2.0$$

To znaczy: te trzy rekordy razem dają odpowiednik dwóch pełnych samochodów szybkich.

### 10.2 Sigma-count

W projekcie **sigma-count** to suma stopni przynależności po punktach próbkowania albo po rekordach, zależnie od kontekstu.

Dla zbioru rozmytego na uniwersum:

$$|A|_{\Sigma} = \sum_{x \in \text{samples}(X)} \mu_A(x)$$

Dla rekordów:

$$\sum_{i=1}^n \mu_S(y_i)$$

Ta druga suma jest podstawą prawdziwości zdań.

### 10.3 Względna kardynalność

Względna kardynalność dzieli licznosc przez rozmiar dziedziny lub liczbe próbek:

$$\text{sigmaRel}(A) = \frac{|A|_{\Sigma}}{|X|}$$

Dzięki temu wynik mieści się w  $[0, 1]$ , więc można porównywać etykiety z różnych zmiennych.

## 11 Podsumowania wielopodmiotowe

### 11.1 Po co więcej niż jeden podmiot

Podsumowania jednopodmiotowe mówią o całym zbiorze samochodów. Podsumowania wielopodmiotowe porównują dwie grupy, np. dwie marki. Zamiast pytać:

*Czy samochody mają dużą moc?*

pytamy:

*Czy samochody marki BMW częściej niż samochody marki Toyota mają dużą moc?*

To jest inny typ wiedzy: nie opis ogólny, lecz porównanie.

### 11.2 Podmioty P1 i P2

W kodzie podmiot to **Subject**. Każdy podmiot ma:

- nazwę do zdania;
- predykat klasyczny sprawdzający, czy rekord należy do grupy.

Dla marek:

```
Subject p1 = Subject.ofMake("BMW");  
Subject p2 = Subject.ofMake("Toyota");
```

Projekt pilnuje, żeby P1 i P2 nie były tym samym podmiotem. Porównywanie BMW z BMW nie ma sensu, więc generator odrzuca takie zdania.

### 11.3 Cztery formy wielopodmiotowe

Projekt implementuje formy I–IV.

| Forma | Schemat                                  | Intuicja                               |
|-------|------------------------------------------|----------------------------------------|
| I     | $QP_1$ w porównaniu do $P_2$<br>jest $S$ | porównanie udziału cechy $S$ w P1 i P2 |

| Forma | Schemat                                                            | Intuicja                                              |
|-------|--------------------------------------------------------------------|-------------------------------------------------------|
| II    | $QP_1$ będących $R$ w porównaniu do $P_2$<br>będących $R$ jest $S$ | porównanie obu grup po tym samym<br>zawężeniu         |
| III   | $QP_1$ będących $R$ w porównaniu do $P_2$<br>jest $S$              | kwalifikator dotyczy tylko P1                         |
| IV    | $QP_1$ w porównaniu do $P_2$<br>jest $S$                           | wariant liczony bez normalizacji po<br>liczności grup |

#### 11.4 Stopień prawdziwości wielopodmiotowej

Dla form I–III projekt liczy najpierw siłę sumaryzatora w każdej grupie. W uproszczeniu:

$$n_1 = \frac{\sum_{y_i \in P_1} \mu_S(y_i)}{|P_1|}$$

$$n_2 = \frac{\sum_{y_i \in P_2} \mu_S(y_i)}{|P_2|}$$

Potem argument kwantyfikatora to:

$$arg = \frac{n_1}{n_1 + n_2}$$

Jeśli  $arg$  jest blisko 1, cecha silnie dominuje w P1. Jeśli jest blisko 0.5, grupy są podobne. Jeśli jest blisko 0, cecha bardziej pasuje do P2.

Dla formy IV projekt używa:

$$arg = \frac{\sum_{y_i \in P_1} \mu_S(y_i)}{\sum_{y_i \in P_1} \mu_S(y_i) + \sum_{y_i \in P_2} \mu_S(y_i)}$$

Forma IV jest bardziej wrażliwa na licznosc grup, bo nie dzieli przez rozmiar podmiotów.

#### 11.5 Kwalifikator w formach II i III

W formie II kwalifikator zawęza obie grupy:

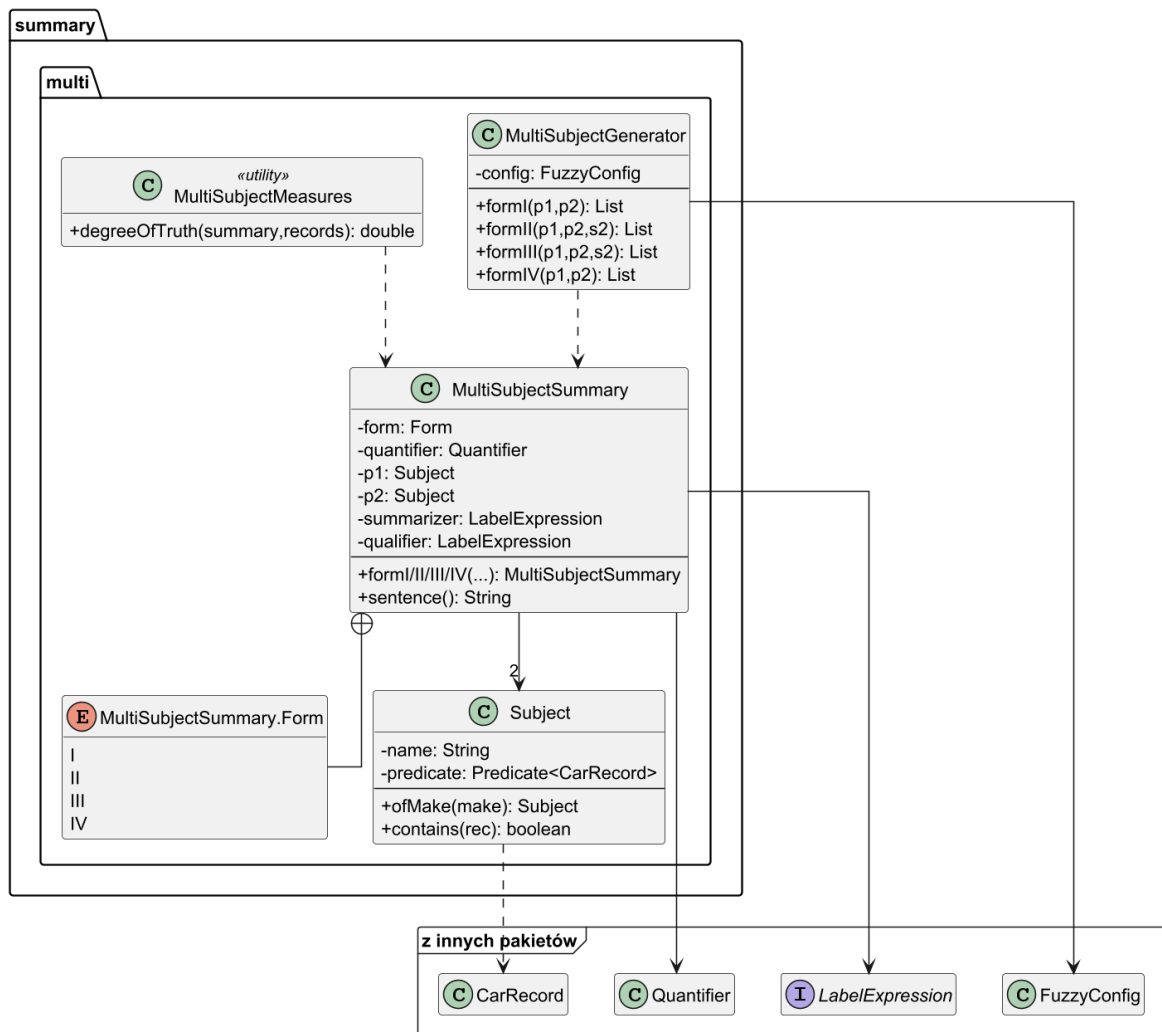
$$P_1 \cap R, \quad P_2 \cap R$$

W formie III kwalifikator zawęza tylko P1:

$$P_1 \cap R, \quad P_2$$

To subtelna, ale ważna różnica. Forma II pyta: „wśród podobnie zawężonych obiektów, która grupa ma cechę bardziej?”. Forma III pyta: „czy zawężona część P1 wypada wobec całego P2 w określony sposób?”.





Rysunek 6: Klasy odpowiedzialne za podsumowania wielopodmiotowe.



## 12.2 Warstwa funkcji przynależności

Interfejs `MembershipFunction` definiuje jedną rzecz: dla wartości liczbowej zwrócić stopień przynależności. Dzięki temu reszta systemu nie musi wiedzieć, czy funkcja jest trójkątna, trapezoidalna czy gaussowska.

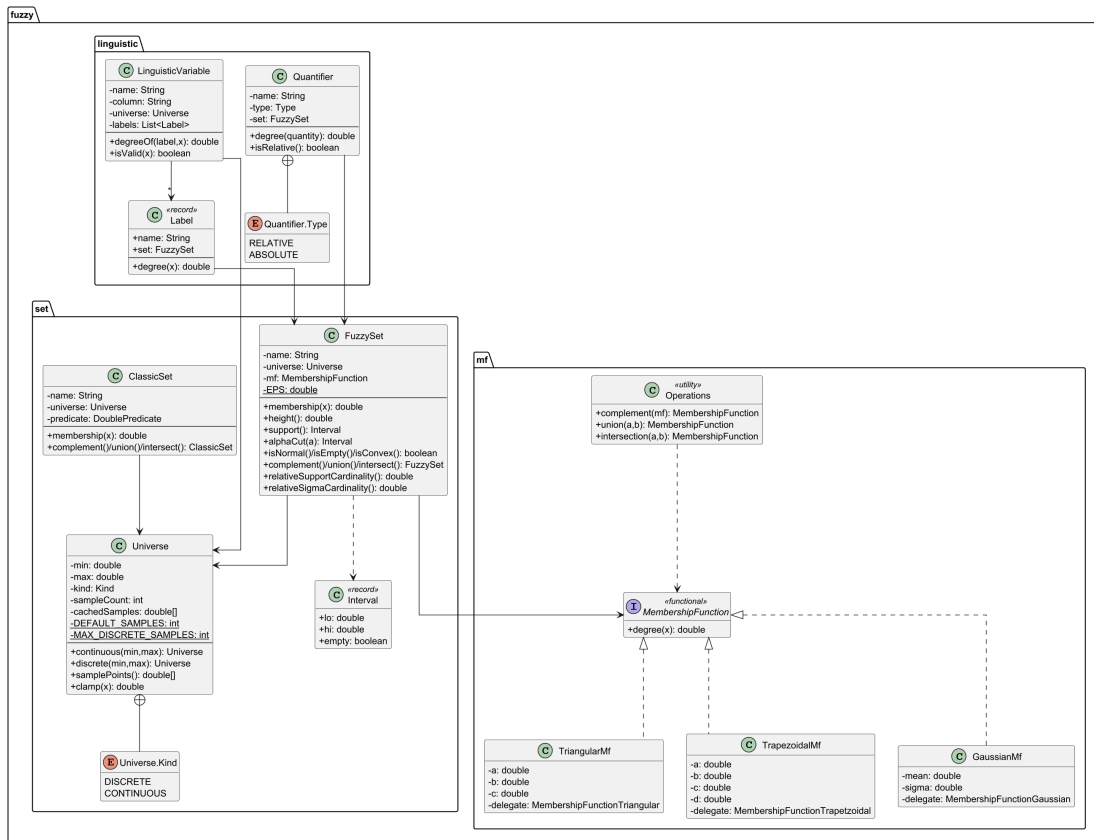
W przybliżeniu:

```
public interface MembershipFunction {  
    double value(double x);  
}
```

To prosta abstrakcja, ale bardzo ważna. Pozwala w trybie zaawansowanym GUI dodawać nowe etykiety z różnymi kształtami funkcji.

## 12.3 Warstwa zbiorów

`FuzzySet` łączy funkcję przynależności z uniwersum. Bez uniwersum funkcja jest tylko wzorem; z uniwersum staje się zbiorem, na którym można policzyć nośnik, kardynalność, normalność i operacje.



Rysunek 8: Warstwa zbiorów rozmytych, uniwersów i funkcji przynależności.

## 12.4 Warstwa konfiguracji

`FuzzyConfig` jest centralnym miejscem konfiguracji. Tworzy:

- listę zmiennych lingwistycznych;
- listę kwantyfikatorów względnych;

- listę kwantyfikatorów absolutnych;
- metody pomocnicze do dodawania własnych etykiet i kwantyfikatorów.

Jeżeli chcesz zmienić znaczenie słowa *duża moc*, to właśnie tutaj szukasz parametrów funkcji przynależności. Jeżeli chcesz dodać nowy kwantyfikator typu *około jedna trzecia*, też zaczynasz od tej warstwy.

## 12.5 Warstwa generowania zdań

`SummaryGenerator` działa na zasadzie kombinatorycznej:

- bierze kwantyfikatory;
- bierze właściwości zbudowane z etykiet;
- składa je w zdania formy I albo II;
- dla złożonych sumaryzatorów używa AND/OR.

Sama składnia zdania jest osobną sprawą. Projekt ma pomocniczą klasę `SentenceGrammar`, która poprawia brzmienie generowanych zdań po polsku, np. dobiera frazy tak, żeby nie brzmiały jak mechanicznie sklejone fragmenty.

## 12.6 Warstwa oceny jakości

`QualityMeasures.evaluate` przechodzi po rekordach i zbiera wartości potrzebne do T1–T11:

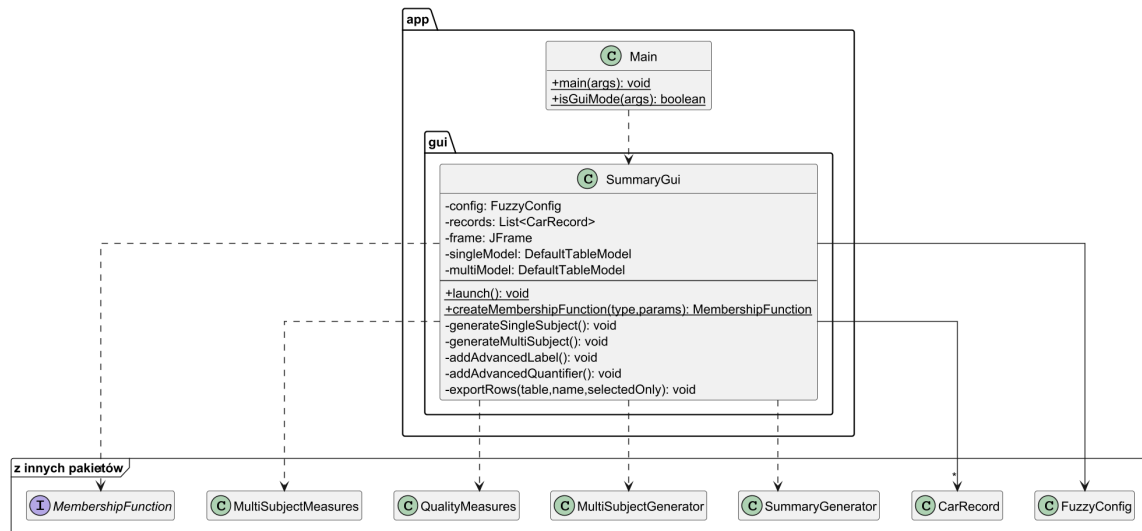
- stopnie sumaryzatora;
- stopnie kwalifikatora;
- liczby rekordów z dodatnią przynależnością;
- rozmyte licznosci;
- nośniki i kardynalności etykiet.

Wynikiem jest obiekt `Quality`, który przechowuje wartości T1–T11 oraz `optimal`.

## 12.7 Warstwa GUI

`SummaryGui` jest aplikacją Swing. Ona nie implementuje teorii od zera, tylko łączy warstwy:

- ładuje dane;
- pozwala użytkownikowi wybrać formę podsumowań;
- uruchamia generator;
- liczy miary;
- pokazuje tabelę;
- eksportuje CSV;
- pozwala dodawać nowe etykiety i kwantyfikatory w trybie zaawansowanym.



Rysunek 9: GUI jako warstwa spinająca dane, konfigurację, generatory i wyniki.

## 13 GUI: jak działa aplikacja

### 13.1 Uruchomienie

Punkt startowy to `Main`. Aplikacja ładuje dane i uruchamia GUI. Jeżeli baza SQLite nie istnieje albo dane wymagają przygotowania, wykorzystywane są komponenty importu i czyszczenia.

Projekt jest Mavenowy, więc typowy workflow programistyczny to:

```
mvn test
mvn exec:java
```

Pierwsza komenda uruchamia testy, druga aplikację.

### 13.2 Zakładka jednopodmiotowa

W tej zakładce użytkownik tworzy podsumowania dla całej bazy samochodów. Może wybrać:

- formę I albo II;
- kwantyfikatory względne albo absolutne;
- summaryzator;
- kwalifikator dla formy II;
- spójnik dla podsumowań złożonych;
- miarę sortowania, np. `OPT` albo `T1`;
- liczbę najlepszych wyników;
- wagi `T1–T11`.

Najważniejsza praktyczna decyzja: sortowanie po `T1` pokaże zdania najbardziej prawdziwe, a sortowanie po `OPT` pokaże zdania najlepsze według wybranego kompromisu wielu miar.

### 13.3 Zakładka wielopodmiotowa

Ta zakładka porównuje dwie grupy, najczęściej dwie marki. Użytkownik wybiera:

- podmiot P1;
- podmiot P2;
- formę I–IV;
- sumaryzator;
- kwalifikator dla form II/III;
- kwantyfikator względny;
- sortowanie rosnące albo malejące po stopniu prawdziwości.

W podsumowaniach wielopodmiotowych kluczowa jest interpretacja argumentu kwantyfikatora. Nie chodzi tylko o to, czy P1 spełnia cechę, ale czy spełnia ją mocniej niż P2.

### 13.4 Tryb zaawansowany

Tryb zaawansowany pozwala rozszerzać system bez zmiany kodu:

- dodać nową etykietę do zmiennej lingwistycznej;
- wybrać funkcję trójkątną, trapezoidalną albo gaussowską;
- podać parametry funkcji;
- dodać nowy kwantyfikator względny;
- dodać nowy kwantyfikator absolutny.

To jest ważne dydaktycznie, bo pozwala zobaczyć, że teoria nie jest zamknięta na sztywno. Granice pojęć językowych są modelowane przez parametry, a parametry można zmieniać i obserwować wpływ na wyniki.

### 13.5 Eksport wyników

GUI pozwala eksportować wyniki do CSV. To umożliwi dalszą analizę poza aplikacją, np. w arkuszu kalkulacyjnym. Eksport ma sens szczególnie wtedy, gdy chcemy porównać rankingi dla różnych wag albo różnych form zdań.

## 14 Jak czytać wygenerowane wyniki

### 14.1 Nie patrz tylko na jedno pole

Najczęstszy błąd to potraktowanie OPT jako jedynej prawdy. OPT jest kompromisem. Jeżeli wagi mocno premiuje krótkość i precyzję, zdanie z niższym T1 może wyjść wysoko.

Rozsądna procedura interpretacji:

1. Sprawdź tekst zdania.
2. Sprawdź T1: czy zdanie jest rzeczywiście prawdziwe w danych?
3. Sprawdź T3: czy dotyczy dużej części danych, czy niszy?

4. Sprawdź T2 i T8: czy sumaryzator jest konkretny?
5. Jeżeli jest kwalifikator, sprawdź T9–T11.
6. Dopiero potem patrz na OPT.

## 14.2 Przykład interpretacyjny

Zdanie:

*Większość samochodów ma małą pojemność silnika.*

Może mieć wysokie T1, jeżeli rozmyta średnia przynależność do etykiety *mała pojemność* trafia w obszar kwantyfikatora *większość*. Ale jeżeli etykieta *mała pojemność* jest bardzo szeroka, T2 i T8 mogą być niższe. Wtedy zdanie jest prawdziwe, ale niekoniecznie bardzo precyzyjne.

Zdanie:

*Prawie żaden samochód ma bardzo dużą pojemność silnika.*

Może być prawdziwe, jeśli takich samochodów jest mało. Ale z punktu widzenia użytkownika warto jeszcze sprawdzić, czy etykieta *bardzo duża pojemność* nie jest zdefiniowana zbyt wąsko. Jeżeli granice etykiety są bardzo ostre lub bardzo przesunięte, wynik może mówić bardziej o konfiguracji niż o danych.

## 14.3 Interpretacja zdań z kwalifikatorem

Zdanie:

*Większość samochodów o dużej mocy silnika ma dużą prędkość maksymalną.*

nie mówi o wszystkich samochodach. Mówi tylko o rozmytej grupie samochodów o dużej mocy. Jeśli kwalifikator obejmuje mało rekordów, T1 może być wysokie, ale T9/T10 powiedzą, że kwalifikator jest bardzo selektywny.

## 14.4 Interpretacja wielopodmiotowa

Zdanie:

*Większość samochodów marki BMW w porównaniu do samochodów marki Toyota ma dużą moc silnika.*

nie oznacza dosłownie, że większość wszystkich BMW ma dużą moc. Oznacza, że w porównaniu rozmytych udziałów między BMW i Toyotą cecha *duża moc* wypada bardziej po stronie BMW zgodnie z kwantyfikatorem.

To jest bardzo ważne: w podsumowaniach wielopodmiotowych kwantyfikator opisuje relację między dwiema grupami, nie zwykły udział w jednej grupie.

## 15 Najważniejsze klasy: przewodnik po kodzie

### 15.1 CarRecord

**CarRecord** reprezentuje jeden rekord samochodu. To obiekt, z którego **Property** pobiera wartości kolumn. Jeżeli w danych wartość jest niepoprawna albo brakująca, dalsze warstwy muszą ją pominąć lub potraktować jako niemożliwą do oceny.

### 15.2 Columns

**Columns** centralizuje nazwy kolumn. Dzięki temu reszta kodu nie musi wpisywać nazw tekstowych w wielu miejscach. To ogranicza ryzyko literówek i ułatwia zmianę źródła danych.

### 15.3 DataCleaner

**DataCleaner** odpowiada za przygotowanie zbioru użytecznego w projekcie. To ważne, bo podsumowania lingwistyczne są wrażliwe na śmieci w danych. Niepoprawne lub brakujące wartości mogłyby zniekształcić stopnie przynależności.

### 15.4 FuzzyConfig

**FuzzyConfig** jest mapą semantyczną projektu. To tutaj surowe kolumny stają się pojęciami językowymi.

Przykład zależności:

$$\text{engine\_hp} \rightarrow \text{moc silnika} \rightarrow \{\text{bardzo mała, mała, średnia, duża, bardzo duża}\}$$

Jeżeli ktoś pyta, gdzie w projekcie zdefiniowano znaczenie słów, odpowiedź brzmi: głównie w **FuzzyConfig**.

### 15.5 Property

**Property** to atom logiczno-lingwistyczny: zmienna plus etykieta. Na przykład:

$$\text{moc silnika} + \text{duża}$$

Dla rekordu samochodu **Property** zwraca stopień, np. 0.73. Generator i miary jakości nie muszą znać szczegółów kolumn ani funkcji przynależności, bo **Property** ukrywa ten mechanizm.

### 15.6 LabelExpression

**LabelExpression** reprezentuje wyrażenie złożone z etykiet. Dzięki temu sumaryzator albo kwalifikator może być traktowany jednolicie niezależnie od tego, czy jest pojedynczy, czy złożony.

### 15.7 SummaryGenerator

**SummaryGenerator** tworzy kandydatów podsumowań. To generator zdań, a nie oceniacz. Jego zadaniem jest poprawnie złożyć kwantyfikator, sumaryzator, ewentualny kwalifikator i tekst.



## 15.8 QualityMeasures

QualityMeasures jest matematycznym centrum oceny. Dla każdego podsumowania przechodzi po rekordach i liczy:

- stopień prawdziwości;
- pokrycie;
- precyzję;
- długość;
- miary kwantyfikatora;
- miary kwalifikatora;
- wynik optymalny.

Jeżeli trzeba debugować „dlaczego to zdanie ma taki wynik”, najczęściej kończy się właśnie tutaj.

## 15.9 MultiSubjectGenerator i MultiSubjectMeasures

Te klasy robią analogiczne rzeczy dla zdań porównujących dwa podmioty:

- generator tworzy zdania form I–IV;
- miary liczą stopień prawdziwości porównania.

Projekt nie liczy pełnego zestawu T1–T11 dla wielopodmiotowych zdań, tylko stopień prawdziwości. To jest zgodne z zakresem implementacji: wielopodmiotowa część skupia się na porównaniu, a nie na pełnym rankingu jakości znanym z jednopodmiotowych podsumowań.

## 15.10 SummaryGui

SummaryGui jest największą klasą interfejsu. Warto czytać ją nie jako teorię, tylko jako orkiestrację:

- tworzy komponenty Swing;
- reaguje na kliknięcia;
- wywołuje generatory;
- pokazuje tabele;
- obsługuje eksport;
- dodaje konfiguracje z trybu zaawansowanego.

## 16 Od zdania do wyniku: pełny przykład

Weźmy zdanie:

*Większość samochodów ma małą pojemność silnika.*

## 16.1 Krok 1: wybór kwantyfikatora

Kwantyfikator:

$$Q = \text{większość}$$

Jest względny, więc jego dziedziną jest  $[0, 1]$ .

## 16.2 Krok 2: wybór sumaryzatora

Sumaryzator:

$$S = \text{mała pojemność silnika}$$

To Property złożone ze zmiennej *pojemność silnika* i etykiety *mała*.

## 16.3 Krok 3: ocena każdego rekordu

Dla każdego samochodu:

1. pobieramy `capacity_cm3`;
2. liczymy  $\mu_S(\text{capacity})$ ;
3. dodajemy wynik do sumy.

Przykładowo:

$$1000 \mapsto 1.0, \quad 1600 \mapsto 0.7, \quad 2200 \mapsto 0.1, \quad 4000 \mapsto 0.0$$

## 16.4 Krok 4: agregacja

Jeżeli mamy  $n$  rekordów:

$$r = \frac{1}{n} \sum_{i=1}^n \mu_S(y_i)$$

To jest rozmyty udział samochodów o małej pojemności.

## 16.5 Krok 5: zastosowanie kwantyfikatora

Stopień prawdziwości:

$$T_1 = \mu_{\text{większość}}(r)$$

Jeżeli  $r = 0.75$ , a kwantyfikator *większość* ma plateau w okolicach 0.70–0.85,  $T_1$  może wynieść 1.0.

## 16.6 Krok 6: pozostałe miary

Następnie liczymy T2–T11:

- T2/T8 z kształtu etykiety *mała pojemność*;
- T3 z liczby rekordów o dodatniej przynależności;
- T5 z długości sumaryzatora;
- T6/T7 z kwantyfikatora *większość*;
- T9–T11 są 0, bo nie ma kwalifikatora.

## 17 Co zmienia konfiguracja funkcji przynależności

### 17.1 Granice słów są decyzją projektową

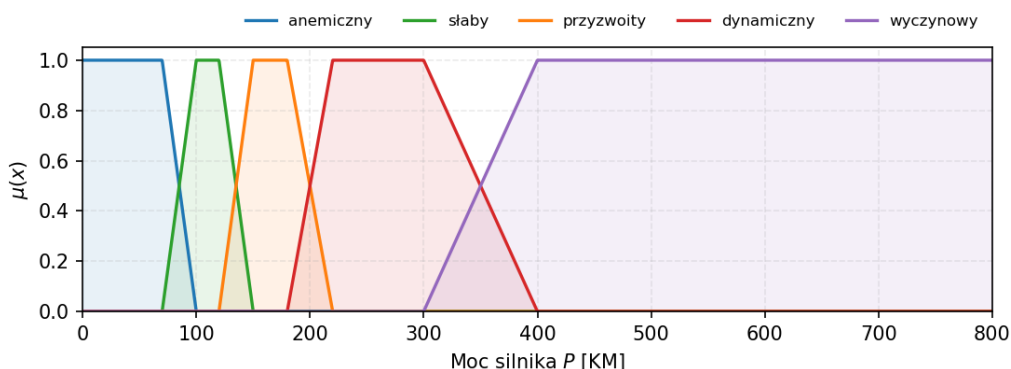
To, czy samochód o mocy 150 KM jest *mocny*, zależy od parametrów etykiety. W tym sensie podsumowania lingwistyczne nie są czystą statystyką. Są statystyką przepuszczoną przez model znaczeń językowych.

Jeżeli przesuniemy etykietę *duża moc* w prawo, mniej samochodów będzie do niej należeć. Zmieni się:

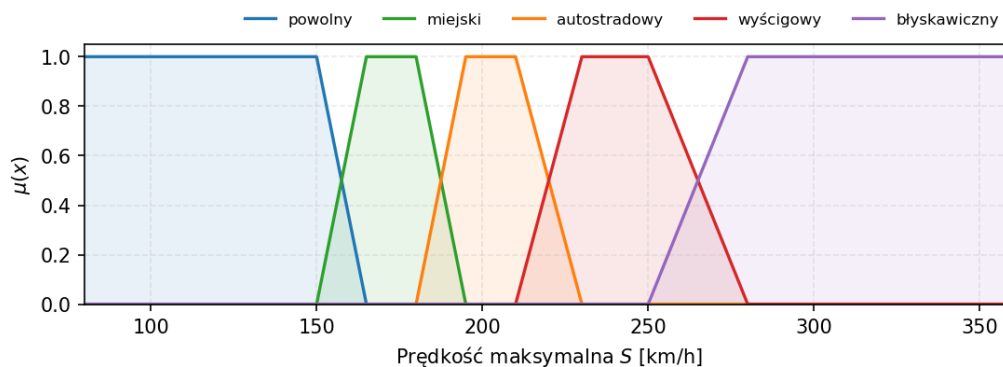
- T1 dla zdań z tą etykietą;
- T3, bo mniej rekordów będzie miało dodatnie pokrycie;
- T2 i T8, bo zmieni się kształt/nośnik/kardynalność etykiety;
- rankingi w GUI.

### 17.2 Dlaczego wykresy są ważne

Wykresy funkcji przynależności pozwalają zobaczyć, co naprawdę znaczą słowa. Bez wykresu łatwo założyć, że *duży* znaczy to samo dla każdego. W systemie znaczy dokładnie tyle, ile opisują parametry funkcji.



Rysunek 10: Etykiety dla mocy silnika.



Rysunek 11: Etykiety dla prędkości maksymalnej.

## 18 Testy i poprawność

### 18.1 Co sprawdzają testy

Testy w projekcie obejmują między innymi:

- uruchamialność głównej aplikacji;
- działanie konfiguracji zaawansowanej;
- komponenty GUI w zakresie możliwym do automatycznego sprawdzenia;
- miary podsumowań wielopodmiotowych.

Testy nie zastępują interpretacji matematycznej, ale chronią przed regresjami: jeżeli zmiana w generatorze albo miarach psuje oczekiwane zachowanie, Maven powinien to wykryć.

### 18.2 Co trzeba sprawdzać ręcznie

Nie wszystko da się łatwo sprawdzić testem jednostkowym:

- naturalność językowa zdań po polsku;
- sens dobranych granic etykiet;
- interpretacyjna przydatność rankingów;
- czy GUI jest ergonomiczne dla użytkownika;
- czy wykresy i UML-e w sprawozdaniu odpowiadają aktualnemu kodowi.

Dlatego pełna ocena projektu wymaga zarówno testów, jak i czytania sprawozdania, kodu oraz wyników działania aplikacji.

## 19 Najczęstsze pułapki

### 19.1 Pomylenie kwantyfikatora względnego z absolutnym

*Większość* działa na proporcji. *Kilka tysięcy* działa na liczbie. Jeżeli podstawimy zły typ argumentu, wynik nie ma sensu.

## 19.2 Traktowanie stopnia przynależności jak prawdopodobieństwa

$\mu_A(x) = 0.7$  nie znaczy, że prawdopodobieństwo przynależności wynosi 70%. Znaczy, że wartość pasuje do pojęcia  $A$  w stopniu 0.7. To jest semantyczna zgodność z pojęciem, nie losowość.

## 19.3 Traktowanie T1 jako jedynej miary jakości

T1 mówi o prawdziwości. Nie mówi pełnej historii o precyzji, długości, selektywności ani informacyjności. Dlatego w tabelach są T1–T11.

## 19.4 Zapominanie o kwalifikatorze

Zdanie z kwalifikatorem mówi o podzbiorze rozmytym. Jeśli kwalifikator jest bardzo wąski, zdanie może być prawdziwe, ale dotyczyć niewielkiej części danych.

## 19.5 Mylenie form wielopodmiotowych

Forma I, II, III i IV nie są kosmetycznymi wariantami tekstu. Różnią się tym, jak liczą porównanie i gdzie stosują kwalifikator.

# 20 Ściąga pojęć

| Pojęcie                   | Znaczenie                                                        |
|---------------------------|------------------------------------------------------------------|
| uniwersum                 | dziedzina wartości, np. zakres mocy silnika                      |
| zbiór rozmyty             | zbiór opisany funkcją przynależności do $[0, 1]$                 |
| funkcja przynależności    | zamienia wartość liczbową na stopień pasowania do pojęcia        |
| etykieta                  | nazwa językowa plus zbiór rozmyty, np. <i>duża moc</i>           |
| zmienna lingwistyczna     | atrybut opisany etykietami, np. <i>moc silnika</i>               |
| kwantyfikikator           | rozmyte określenie liczności lub proporcji, np. <i>większość</i> |
| kwantyfikikator względny  | działa na proporcji $[0, 1]$                                     |
| kwantyfikikator absolutny | działa na liczności                                              |
| sumaryzator               | cecha przypisywana obiektom w zdaniu                             |
| kwalifikator              | cecha zawężająca zbiór obiektów                                  |
| podmiot                   | zbiór obiektów, np. wszystkie samochody albo marka BMW           |
| T1                        | stopień prawdziwości zdania                                      |
| OPT                       | średnia ważona T1–T11                                            |

# 21 Ściąga po klasach

| Klasa      | Rola                   |
|------------|------------------------|
| Main       | uruchamia aplikację    |
| SummaryGui | interfejs użytkownika  |
| CarRecord  | jeden rekord samochodu |

| Klasa                 | Rola                                   |
|-----------------------|----------------------------------------|
| CarDataLoader         | ładuje rekordy z bazy/danych           |
| DataCleaner           | przygotowuje oczyszczony zbiór danych  |
| Universe              | dziedzina dla zbioru rozmytego         |
| FuzzySet              | zbiór rozmyty i operacje na nim        |
| ClassicSet            | klasyczny predykat zbioru              |
| MembershipFunction    | interfejs funkcji przynależności       |
| TriangularMf          | funkcja trójkątna                      |
| TrapezoidalMf         | funkcja trapezoidalna                  |
| GaussianMf            | funkcja gaussowska                     |
| Label                 | etykieta lingwistyczna                 |
| LinguisticVariable    | zmienna lingwistyczna                  |
| Quantifier            | kwantyfikator względny lub absolutny   |
| Property              | atom summaryzatora/kwalifikatora       |
| Connective            | spójniki AND/OR                        |
| SummaryGenerator      | generator podsumowań jednopodmiotowych |
| SingleSubjectSummary  | obiekt zdania jednopodmiotowego        |
| QualityMeasures       | obliczanie T1–T11                      |
| QualityWeights        | wagi do wyniku optymalnego             |
| Subject               | podmiot w porównaniu wielopodmiotowym  |
| MultiSubjectGenerator | generator zdań wielopodmiotowych       |
| MultiSubjectMeasures  | prawdziwość zdań wielopodmiotowych     |

## 22 Jak wytłumaczyć projekt komuś na głos

Jeżeli masz opowiedzieć ten projekt na zajęciach albo obronić jego sens, dobry schemat wypowiedzi jest taki:

1. Mamy bazę samochodów i chcemy generować z niej zdania opisowe, a nie tylko statystyki.
2. Ponieważ pojęcia typu *duży*, *mały*, *szybki* są nieostre, używamy logiki rozmytej.
3. Każda zmienna liczbową ma etykiety lingwistyczne opisane funkcjami przynależności.
4. Kwantyfikatory, takie jak *większość*, też są zbiorami rozmytymi.
5. Generator składa zdania z kwantifikatora, podmiotu, summaryzatora i opcjonalnego kwalifikatora.
6. Miara T1 mówi, na ile zdanie jest prawdziwe w danych.
7. Miary T2–T11 oceniają dodatkowo precyzję, pokrycie, długość, kwantyfikator i kwalifikator.
8. GUI pozwala wybierać formy zdań, sortować wyniki, zmieniać wagi i dodawać własne etykiety.
9. Wersja wielopodmiotowa porównuje dwie grupy, np. marki samochodów.
10. Całość jest zaimplementowana warstwowo: dane, logika rozmyta, konfiguracja lingwistyczna, generatory, miary, GUI.

## 23 Bibliografia orientacyjna

Ten dokument korzysta z teorii i materiałów opisanych w sprawozdaniu projektu, w szczególności z klasycznego podejścia do podsumowań lingwistycznych baz danych i logiki rozmytej:

- L. A. Zadeh, prace o zbiorach rozmytych, zmiennych lingwistycznych i kwantyfikatorach rozmytych.
- R. R. Yager, koncepcja lingwistycznych podsumowań danych.
- J. Kacprzyk, R. R. Yager, S. Zadrozny, prace o podsumowaniach lingwistycznych baz danych.
- A. Niewiadomski, materiały wykładowe i publikacje dotyczące podsumowań lingwistycznych.
- Dokumentacja i kod projektu `linguistic-summaries`.

## Zakończenie

Cały projekt można streścić jednym zdaniem: jest to system, który modeluje nieostre pojęcia językowe matematycznie, stosuje je do rekordów samochodów, generuje zdania opisujące dane i ocenia te zdania wieloma miarami jakości.

Najważniejsze jest zrozumienie mostu między językiem a matematyką:

słowo  $\rightarrow$  etykieta  $\rightarrow$  funkcja przynależności  
 $\rightarrow$  stopnie dla rekordów  $\rightarrow$  podsumowanie  $\rightarrow$  miary jakości

Gdy ten łańcuch jest jasny, reszta kodu staje się dużo mniej tajemnicza: klasy nie są przypadkowymi elementami aplikacji, tylko kolejnymi poziomami tej samej idei.